

Tutorial on Ontop Protégé

Ontop is an open-source Ontology Based Data Access (OBDA) system that allows for querying relational data sources through a conceptual representation of the domain of interest, provided in terms of an ontology, to which the data sources are mapped.

Ontop is a platform to query databases such as Virtual RDF Graphs using SPARQL and exploit ontologies and mappings. It's extremely fast and is packed with features.

The tutorial will help you:

1. **Define an Ontology in Protégé:**
 - Create or load an ontology in Protégé.
2. **Make use of an existing H2 database.**
 - H2 is an open-source lightweight Java relational database. It can be embedded in Java applications or run in the client-server mode.
 - In order to use H2 Database, you need to have the following components
 - A web browser
 - A H2 console server
 - Have a relational database with tables and data (e.g., a MySQL database).
3. **Create Mappings:**
 - Use the Ontop Protégé plugin to define mappings between the database schema and the ontology.
 - For example, map a table column to an ontology property.
4. **Query the Database Using SPARQL:**
 - Once mappings are defined, you can write SPARQL queries in Protégé or through an external SPARQL endpoint provided by Ontop.
 - Ontop translates the SPARQL query into SQL, executes it on the relational database, and returns the results as RDF triples.

Follow the step-by-step guide

1. Database Setup

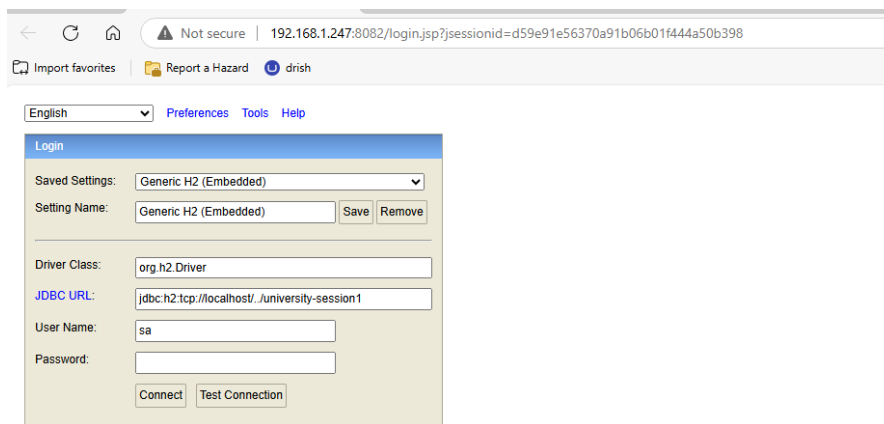
1. Download all files provided under Week 5 ‘Lab Materials’, save it on your local folder and ensure you know how to access this. Unzip the files.

 h2
 ontop-protege-bundle-win-5.2.1
 university.obda
 university.properties
 university.ttl

2. Open the h2 folder, go into *h2/bin*, and click on the executable *h2w.bat* file.

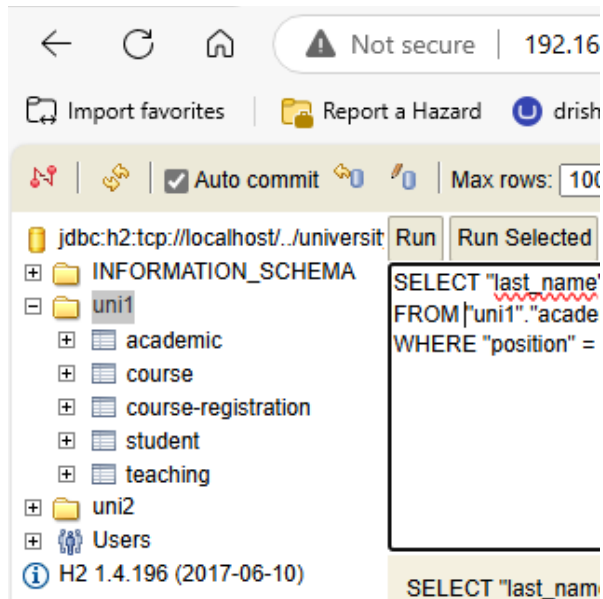
On Mac/Linux: open a terminal, go into *h2/bin* and run `sh h2.sh`

3. After being automatically redirected to the web interface of H2, connect with the default parameters:
 - a. JDBC URL: *jdbc:h2:tcp://localhost/./university-session1*
 - b. User name: *sa*
 - c. No password



The screenshot shows a web browser window with the address bar displaying "192.168.1.247:8082/login.jsp?jsessionid=d59e91e56370a91b06b01f444a50b398". The page has a navigation bar with "English", "Preferences", "Tools", and "Help". The main content area is titled "Login" and contains a "Saved Settings" dropdown menu set to "Generic H2 (Embedded)". Below this is a "Setting Name" field with the same text and "Save" and "Remove" buttons. The "Driver Class" field is set to "org.h2.Driver". The "JDBC URL" field is set to "jdbc:h2:tcp://localhost/./university-session1". The "User Name" field is set to "sa". The "Password" field is empty. At the bottom are "Connect" and "Test Connection" buttons.

Now you can see the tables in the schema *uni1*.



You can check the different tables to familiarize yourself with the H2 database. For example:

4. Try a first SQL query: "Give me the last names of the full professors"

```
SELECT "last_name"  
FROM "uni1"."academic"  
WHERE "position" = 1
```

Position=1 is for full professors

Output SQL query when you also add the first name:

```
SELECT "last_name", "first_name"
FROM "uni1"."academic"
WHERE "position" = 1;
```

last_name	first_name
Chambers	Anna
Josephina	Kyler

(2 rows, 15 ms)

We will come back later for the H2 database, let's now look at the Ontop Setup.

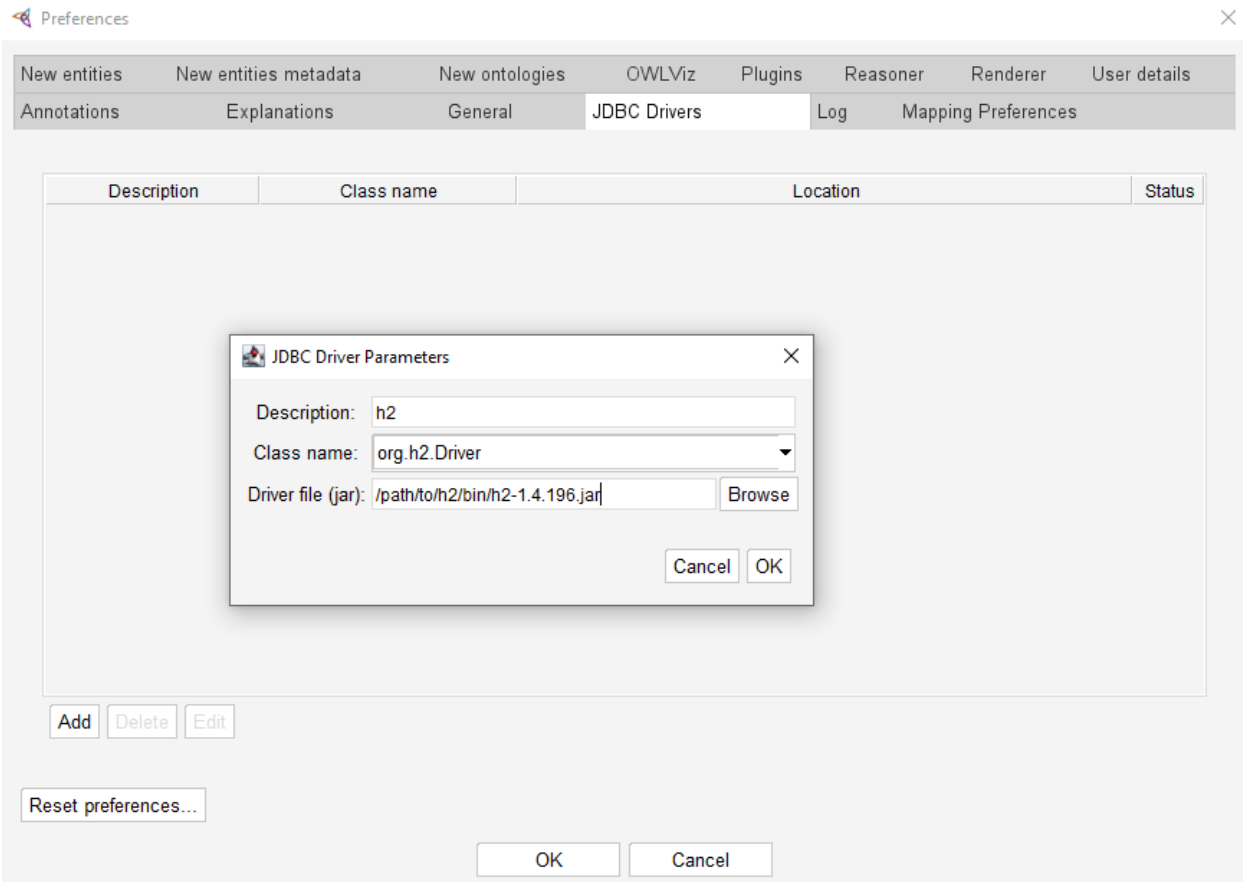
2. Ontop-Protégé setup

Protégé is an open-source ontology editor and knowledge management system. Ontop-Protégé is a plugin for designing and testing a VKG (Virtual Knowledge Graphs) specification.

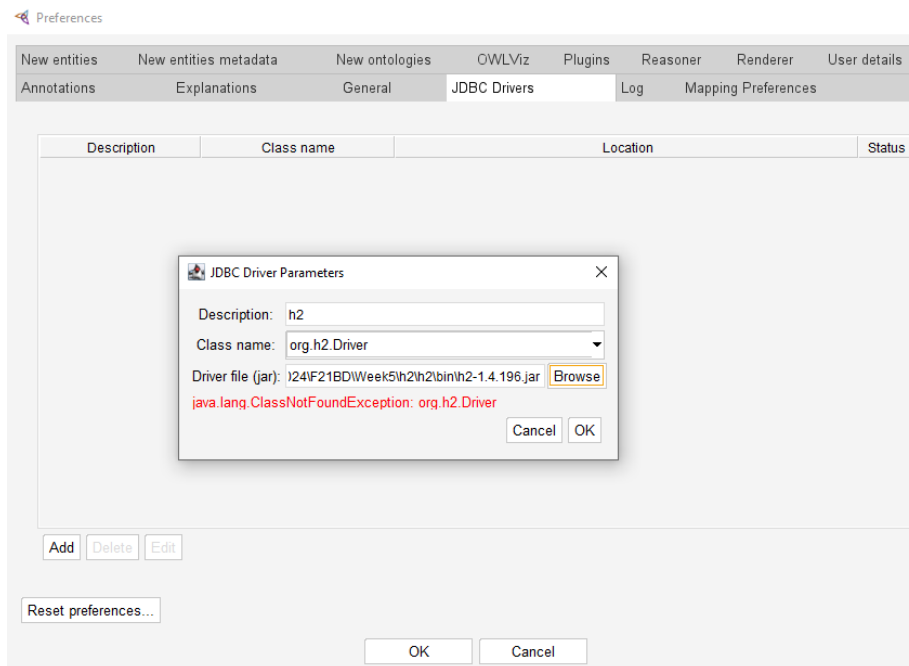
1. [Download the latest Ontop-Protégé bundle for your Operating System \(opens new window here\)](#) and unzip the 'ontop-protége-bundle-win-5.2.1' and go into its folder. The Windows version has already been provided to you in the previously downloaded folder 'Week 5 lab' folder.

 h2	
 ontop-protége-bundle-win-5.2.1	
 h2	
 ontop-protége-bundle-win-5.2.1	
 university.obda	
 university.properties	
 university.ttl	

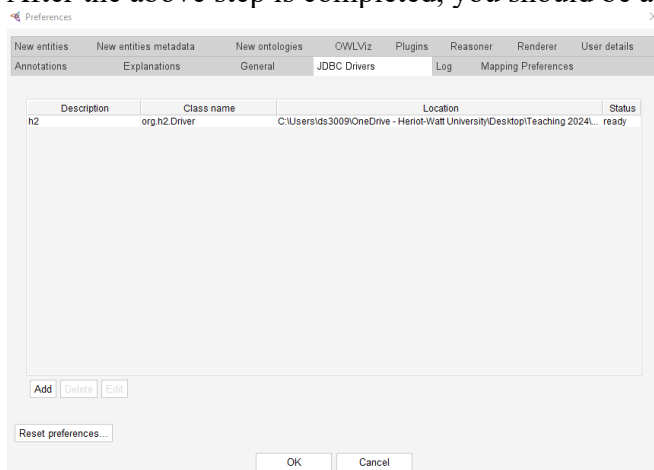
2. Run the Windows Batch File ---- *run.bat* on Windows, *run.sh* on Mac/Linux)
3. Register the H2 JDBC driver: go to File>Preferences -> JDBC Drivers>Add and add an entry with the following information
 1. Description: *h2*
 2. Class Name: *org.h2.Driver*
 3. Driver file (jar): */path/to/h2/bin/h2-1.4.196.jar*
(to avoid an error, click on Browse and look for the **h2-1.4.196.jar** file from the **bin** folder of **H2** folder)



Ensure you add the path where you saved your H2 files, otherwise you will get an error as below:

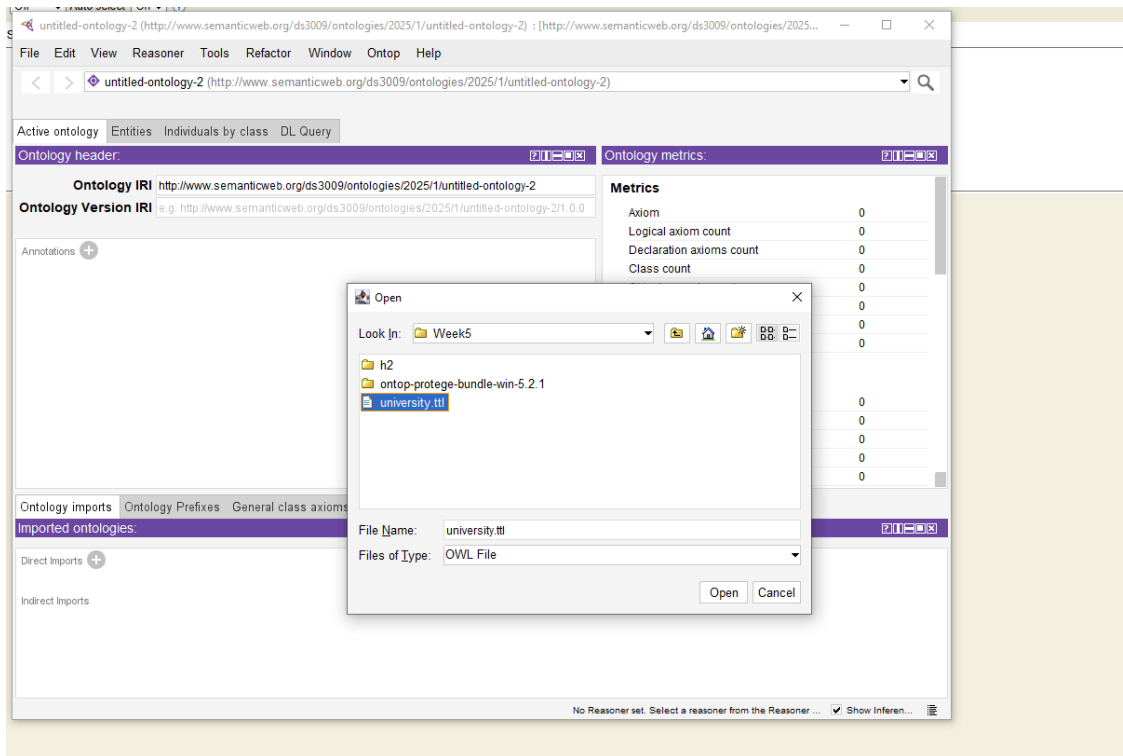


After the above step is completed, you should be able to see this.

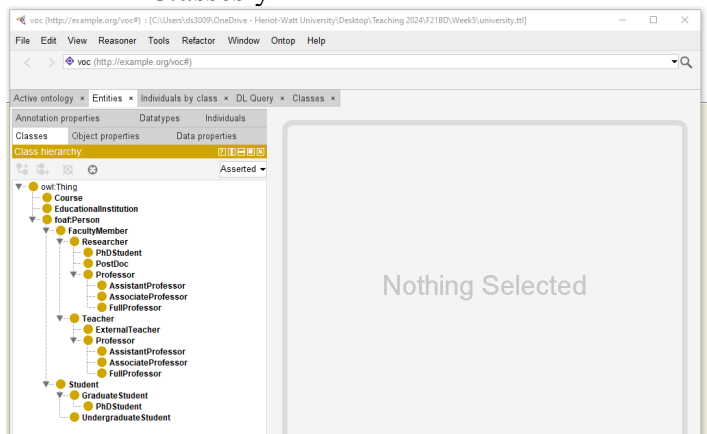


Click on OK, congratulations, you have set up the JDBC drivers correctly. Now let's look at the ontology, the classes and the properties. Before carrying out any mapping it is good to understand the data from H2.

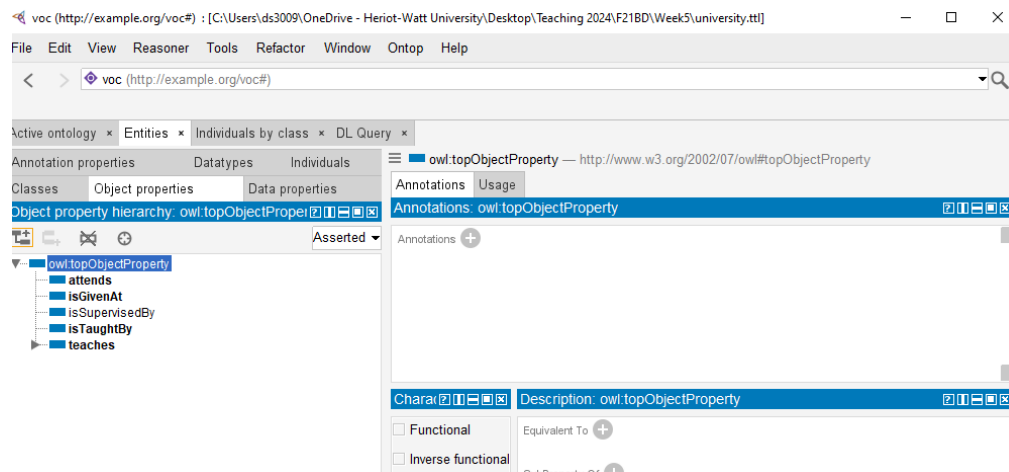
4. In Protégé, go to *File/Open...* to load the ontology OWL file (university.ttl from Week 5 lab Folder).



5. In the Tab class, go to Entities and expand the Classes under the Class Hierarchy. In the tab *Classes* you can visualize the class hierarchy.



6. In the tab *Object properties*, you can see the properties *attends*, *isGivenAt*, *isSupervisedBy*, *isTaughtBy* and *teaches* (with its two sub-properties *givesLab* and *givesLecture*).

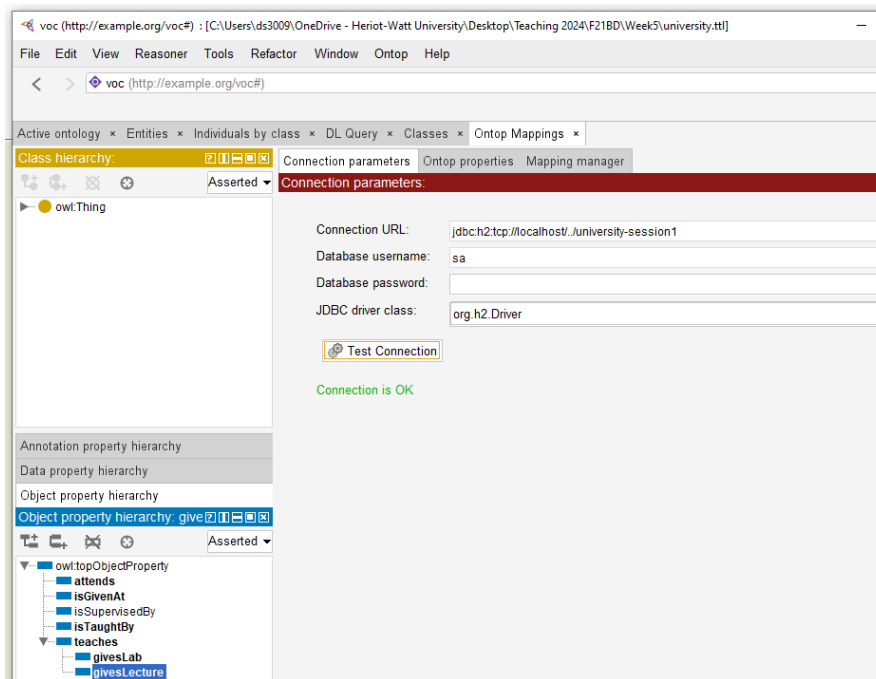


7. In the tab *Data properties* you can see the properties *firstName*, *lastName* and *title*
- ▼- **owl:topDataProperty**
 - **foaf:firstName**
 - **foaf:lastName**
 - **title**

Now we will do the mapping.

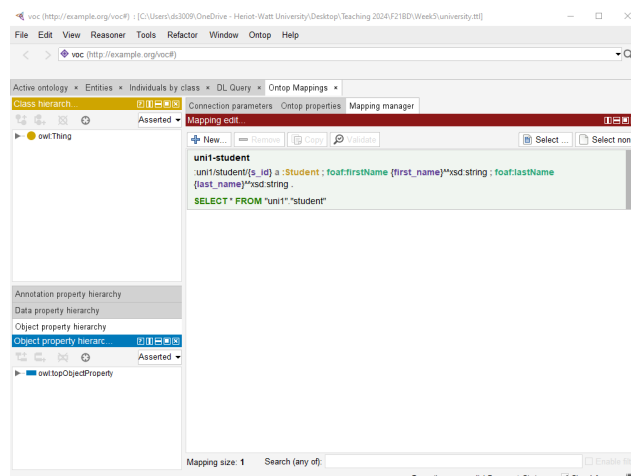
3. Mapping

1. Go to the *Window* -> *Tabs* -> and check (✓) *Ontop mapping* tab. You will now see a new tab *Ontop Mappings* add, click on it.



Connection URL: jdbc:h2:tcp://localhost/./university-session1

2. Test the already defined connection configuration using the *Test Connection* button. Use the above details to connect.
3. Switch to the *Mapping Manager* tab in the ontop mappings tab
4. You should see a first mapping assertion called *uni1-student*
5. Double-click on it to observe the Target and the Source and then close this pop-up window.



Mapping uni1-student

- Target:

```
:uni1/student/{s_id} a :Student ;
foaf:firstName {first_name}^^xsd:string ;
foaf:lastName {last_name}^^xsd:string .
```

- Source:

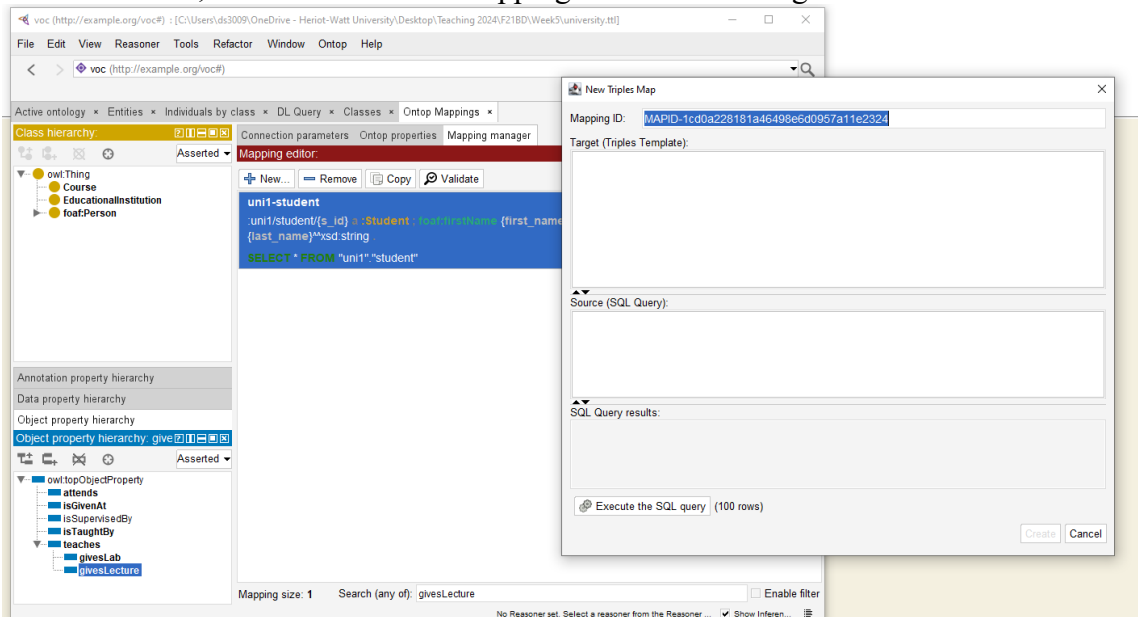
```
SELECT *
FROM "uni1"."student"
```

The target part is described using a Turtle-like syntax while the source part is a regular SQL query.

Let us now add the other mapping assertions by clicking on *create*:

Your task:

Click on +New, then create 5 more mappings for the following:



The screenshot shows the Protégé ontology editor interface. On the left, there is a class hierarchy with 'Course' and 'EducationalInstitution' as subclasses of 'owl:Thing'. The 'Mapping editor' tab is active, showing a mapping for 'uni1-student' with a target URI and a source SQL query. A 'New Triples Map' dialog box is open in the foreground, allowing the user to create a new mapping. The dialog box contains a 'Mapping ID' field (MAPIID-1cd0a228161a46498e6d0957a11e2324), a 'Target (Triples Template)' field, a 'Source (SQL Query)' field, and a 'SQL Query results' field. The 'Mapping ID' field is highlighted. The dialog box also has 'Create' and 'Cancel' buttons at the bottom right.

Create Mapping for

Rename your Mapping in the MappingID (uni1-academic, uni1-course, uni1-teaching etc) if you wish to make it easier to follow.

Mapping uni1-academic

- **Target:**
:uni1/academic/{a_id} a :FacultyMember ;
foaf:firstName {first_name}^^xsd:string ;
foaf:lastName {last_name}^^xsd:string .
- **Source:**
SELECT *
 - FROM "uni1"."academic"

Mapping uni1-course

- **Target:**
:uni1/course/{c_id} a :Course ;
:title {title} ;
:isGivenAt :uni1/university .
- **Source:**
SELECT *
 - FROM "uni1"."course"

Mapping uni1-teaching

- **Target:**
:uni1/academic/{a_id} :teaches :uni1/course/{c_id} .
- **Source:**
SELECT *
 - FROM "uni1"."teaching"

Mapping uni1-registration

- **Target:**
:uni1/student/{s_id} :attends :uni1/course/{c_id} .
- **Source:**
SELECT *
 - FROM "uni1"."course-registration"

Mapping uni1-fullProfessor

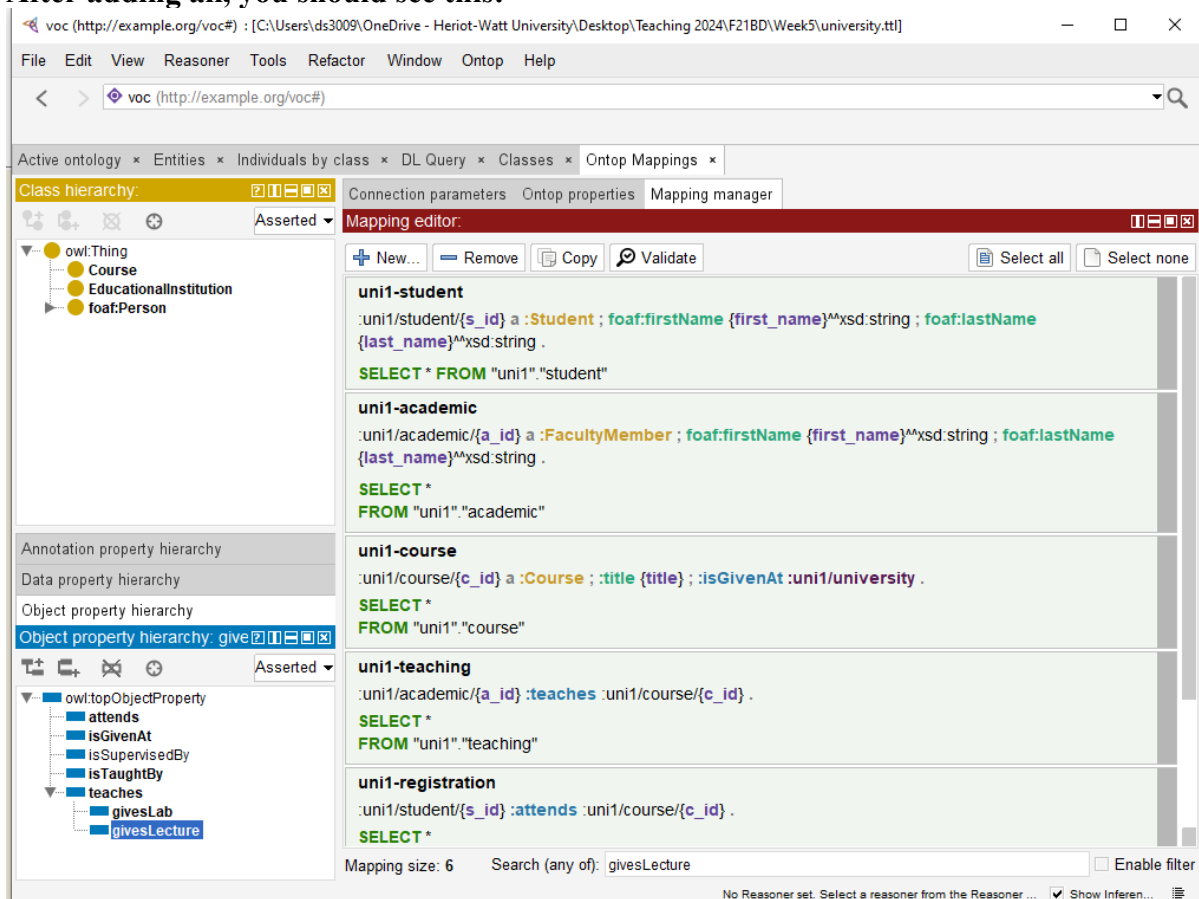
- **Target:**

:uni1/academic/{a_id} a :FullProfessor .

- **Source:**

```
SELECT *
FROM "uni1"."academic"
WHERE "position" = 1
```

After adding all, you should see this:



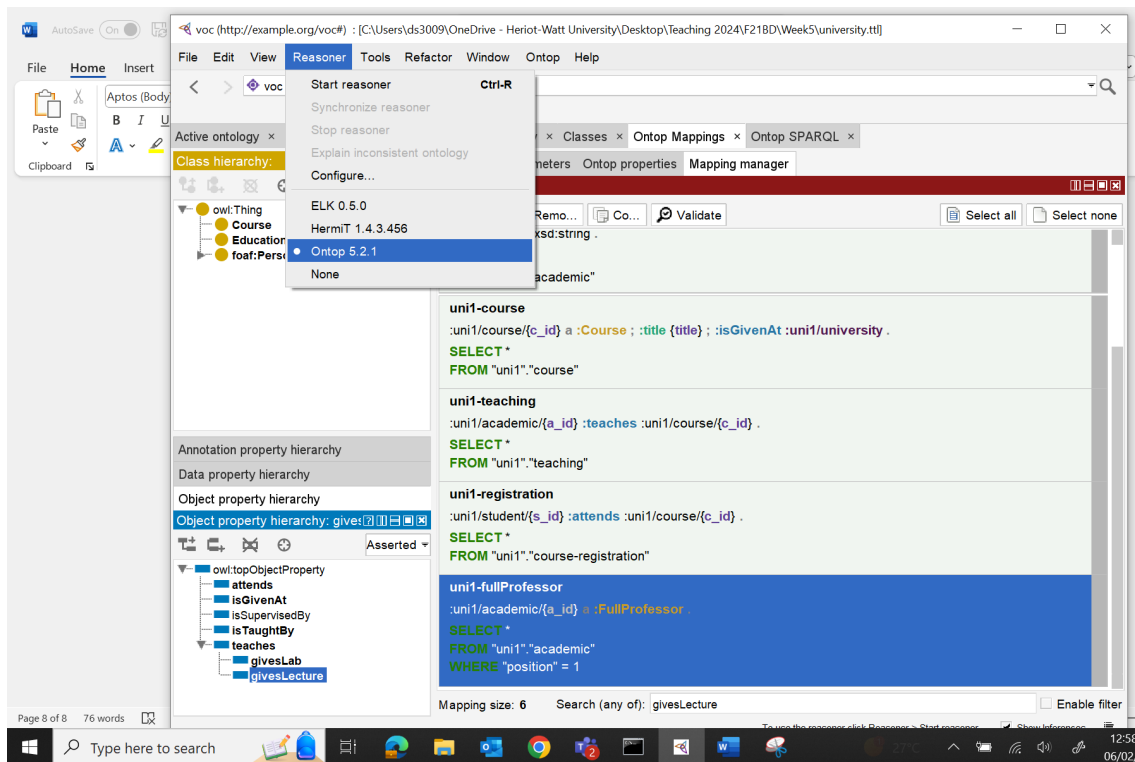
The screenshot shows the Ontop Mapping Editor interface. The left sidebar displays the Class hierarchy (owl:Thing, Course, EducationalInstitution, foaf:Person) and the Object property hierarchy (owl:topObjectProperty, attends, isGivenAt, isSupervisedBy, isTaughtBy, teaches, givesLab, givesLecture). The main area shows the Mapping editor with a list of mappings:

- uni1-student**: :uni1/student/{s_id} a :Student ; foaf:firstName {first_name}^xsd:string ; foaf:lastName {last_name}^xsd:string .
SELECT * FROM "uni1"."student"
- uni1-academic**: :uni1/academic/{a_id} a :FacultyMember ; foaf:firstName {first_name}^xsd:string ; foaf:lastName {last_name}^xsd:string .
SELECT * FROM "uni1"."academic"
- uni1-course**: :uni1/course/{c_id} a :Course ; :title {title} ; :isGivenAt :uni1/university .
SELECT * FROM "uni1"."course"
- uni1-teaching**: :uni1/academic/{a_id} :teaches :uni1/course/{c_id} .
SELECT * FROM "uni1"."teaching"
- uni1-registration**: :uni1/student/{s_id} :attends :uni1/course/{c_id} .
SELECT *

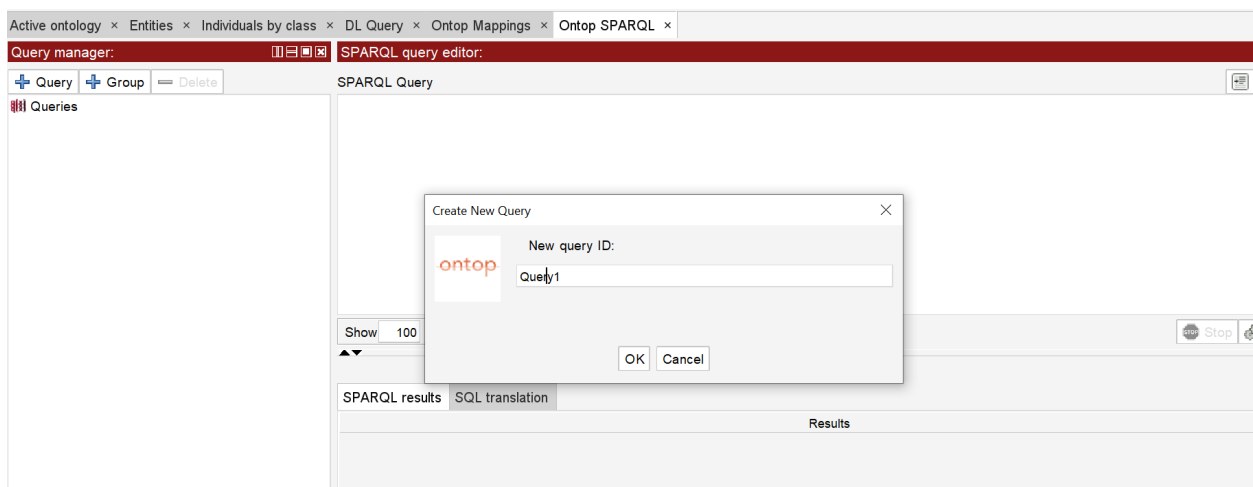
The Mapping size is 6. Search (any of): givesLecture. Enable filter is checked. No Reasoner set. Select a reasoner from the Reasoner ...

SPARQL

1. Run Protégé and go to the *Window -> Tabs -> Ontop SPARQL* tab
2. Select Ontop in the *Reasoner* menu
3. Start the reasoner

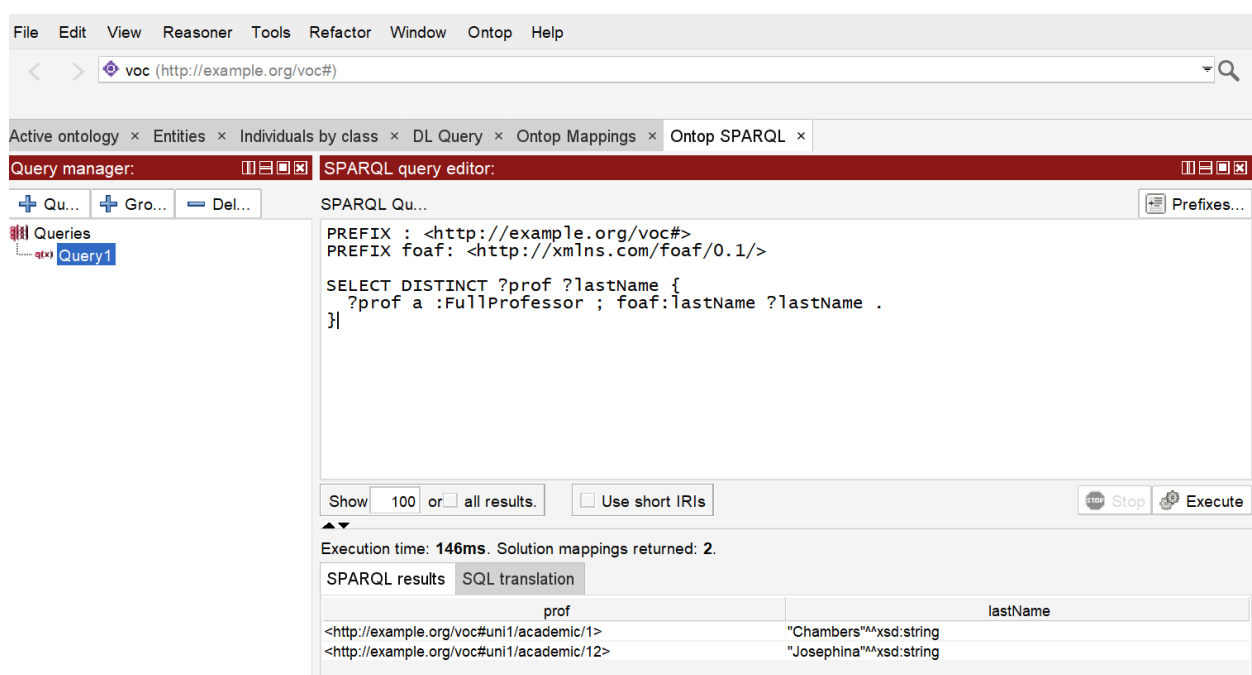


4. Add a query in the *Query Manager* and run the following query in *SPARQL query editor*:



```
PREFIX : <http://example.org/voc#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

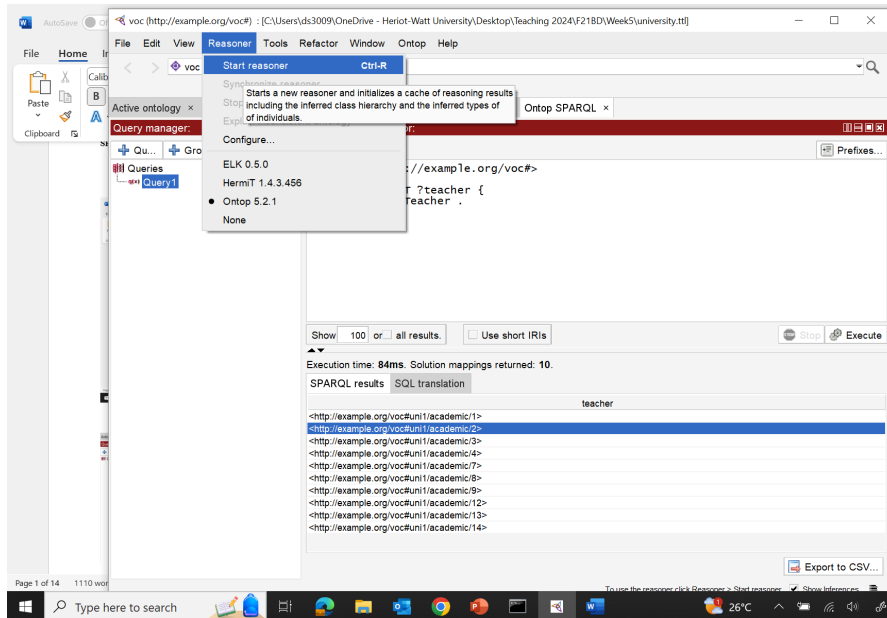
SELECT DISTINCT ?prof ?lastName {
  ?prof a :FullProfessor ; foaf:lastName ?lastName .
}
```



The screenshot shows the Ontop web interface. The top menu bar includes File, Edit, View, Reasoner, Tools, Refactor, Window, Ontop, and Help. The address bar shows the active ontology: `voc (http://example.org/voc#)`. The main workspace is titled "SPARQL query editor:" and contains the same query as shown in the previous block. Below the editor, there are buttons for "Show", "100", "or all results.", and "Use short IRIs". To the right are "Stop" and "Execute" buttons. The results section shows "Execution time: 146ms. Solution mappings returned: 2." and a table with two columns: "prof" and "lastName".

prof	lastName
<code><http://example.org/voc#uni1/academic/1></code>	<code>"Chambers""xsd:string</code>
<code><http://example.org/voc#uni1/academic/12></code>	<code>"Josephina""xsd:string</code>

Ensure you start the reasoner in case you get an error:



Conclusion:

You can see the results are the same as when you did a simple query in H2 database at the beginning of the tutorial. Here we can see 2 last names of the academics who have a 'Full Professor' rank. In this tutorial you have designed a Virtual Knowledge Graph (VKG) specification and you have used the Ontop SPARQL to retrieve some info. Once your mappings and your ontology are ready, you can deploy your VKG as a SPARQL endpoint.

Ontop is a Virtual Knowledge Graph system. It exposes the content of arbitrary relational databases as knowledge graphs. These graphs are virtual, which means that data remains in the data sources instead of being moved to another database.

Further work using second data source: University 2:

<https://ontop-vkg.org/tutorial/basic/university-2.html#new-mappings>

References: <https://ontop-vkg.org/tutorial/basic/setup.html#database-setup>